COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	October 1, 2026	**DATE SUBMITTED:**	October 6, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 GRAPHS

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1 Create a Graph

Following are the basic operations that we can perform on the graph data structure:

- **Add a vertex:** Adds vertex to the graph.
- **Add an edge:** Adds an edge between the two vertices of a graph.
- **Display the graph vertices:** Display the vertices of a graph.

Implement the given code and indicate your output in the table.
(Table 10-1. Create a Graph)

Screenshot of Code	**My Code: (Graph.h)** ILOA > C Graph.h > displayMatrix() 1 #ifndef GRAPH_H 2 #define GRAPH_H 3 4 #include <iostream> 5 6 class Graph{ 7 private: 8 static const int maxVertices = 10; 9 char vertices[maxVertices]; 10 int matrix[maxVertices][maxVertices]; 11 int vertexCount; 12 13 public: 14 Graph();

```
15          void addVertex(char vertex);
16          int findVertex(char vertex);
17          void addEdge(char start, char end);
18          void addWeightedEdge(char start, char end, int weight);
19          void displayVertices();
20  };
21
22  // Create a new graph
23  Graph::Graph(){
24      vertexCount = 0;
25
26      // Initialize the adjacency matrix
27      for(int i = 0; i < maxVertices; i++){
28          for(int j = 0; j < maxVertices; j++){
29              matrix[i][j] = 0;
30          }
31      }
32  }
33
34  // Add a vertex
35  void Graph::addVertex(char vertex){
36
37      // Check if there is space for a new vertex
38      if(vertexCount < maxVertices){
39          vertices[vertexCount] = vertex;
40          vertexCount++;
```

```
41      }
42  }
43
44  // Find the position of a vertex
45  int Graph::findVertex(char vertex){
46
47      // Search the vertices
48      for(int i = 0; i < vertexCount; i++){
49          if(vertices[i] == vertex){
50              return i;
51          }
52      }
53
54      return -1;
55  }
56
57  // Add an undirected edge
58  void Graph::addEdge(char start, char end){
59      int startIndex = findVertex(start);
```

```
60      int endIndex = findVertex(end);
61
62      // Check if both vertices exist
63      if(startIndex != -1 && endIndex != -1){
64          matrix[startIndex][endIndex] = 1;
65          matrix[endIndex][startIndex] = 1;
66      }
67  }
68
69  // Add a weighted directed edge
70  void Graph::addWeightedEdge(char start, char end, int weight){
71      int startIndex = findVertex(start);
72      int endIndex = findVertex(end);
73
74      // Check if both vertices exist
75      if(startIndex != -1 && endIndex != -1){
76          matrix[startIndex][endIndex] = weight;
77      }
78  }
79
80  // Display the graph vertices
81  void Graph::displayVertices(){
82
```

```
83      // Display all vertices
84      for(int i = 0; i < vertexCount; i++){
85          std::cout << vertices[i] << " ";
86      }
87
88      std::cout << std::endl;
89  }
```

My Code: (main_ILOA.cpp)

```
ILOA > main_ILOA.cpp > main()
1   #include <iostream>
2   #include "Graph.h"
3
4   int main(){
5
6       // CREATE A GRAPH
7       std::cout << "\nCreate a Graph:" << std::endl;
8       std::cout << "-----------------------------------" << std::endl;
9
```

```cpp
    // Create a new graph
    Graph graph;

    // Add the vertices
    graph.addVertex('A');
    graph.addVertex('B');
    graph.addVertex('C');
    graph.addVertex('D');
    graph.addVertex('E');

    // Add the edges
    graph.addEdge('A', 'B');
    graph.addEdge('A', 'C');
    graph.addEdge('A', 'D');
    graph.addEdge('B', 'E');
    graph.addEdge('C', 'D');
    graph.addEdge('D', 'E');

    // Display the graph vertices
    std::cout << "Graph Vertices: ";
    graph.displayVertices();

    // Display the graph
    std::cout << "\nGraph:" << std::endl;
    graph.displayMatrix();

```

Output

```
Create a Graph:
----------------------------------------
Graph Vertices: A B C D E

Graph:
    A   B   C   D   E
A   0   1   1   1   0
B   1   0   0   0   1
C   1   0   0   1   0
D   1   0   1   0   1
E   0   1   0   1   0
```

Observations	The code successfully created the graph using the ***Graph class*** with an ***adjacency matrix*** to store the connections between vertices. The ***addVertex()*** function was used to add each vertex, while the ***addEdge()*** function connected the vertices by setting the corresponding matrix values to ***1***. The ***findVertex()*** function was used to find the position of each vertex before adding an edge, and the ***displayVertices()*** function displayed all the added vertices. In ***main_ILOA.cpp***, the vertices ***A, B, C, D,*** and ***E*** were created and connected based on the given edges, such as ***A to B, C,*** and ***D, B to E, C to D,*** and ***D to E***. The output showed all the vertices and the ***adjacency matrix*** with the correct connections between them. Overall, the ***addVertex(), addEdge()***, and ***findVertex()*** functions worked together to create the required graph structure correctly.

Table 10-1. Create a Graph

A.2. Adjacency Matrix

Implement the given code and indicate your output in the table.
(Table 10-2. Adjacency Matrix of Undirected Graph and Weighted Graph)

Screenshot of Code	**My Code: (Graph.h)**
	92 // Display the adjacency matrix
	93 void Graph::displayMatrix(){
	94
	95 // Display the column vertices
	96 std::cout << " ";
	97
	98 for(int i = 0; i < vertexCount; i++){
	99 std::cout << vertices[i] << " ";
	100 }
	101
	102 std::cout << std::endl;
	103
	104 // Display the matrix values
	105 for(int i = 0; i < vertexCount; i++){
	106 std::cout << vertices[i] << " ";
	107

```
108             for(int j = 0; j < vertexCount; j++){
109                 std::cout << matrix[i][j] << "    ";
110             }
111
112             std::cout << std::endl;
113         }
114     }
```

My Code: (main_ILOA.cpp)

```
36      // ADJACENCY MATRIX OF UNDIRECTED GRAPH
37      std::cout << "\nAdjacency Matrix of Undirected Graph:" << std::endl;
38      std::cout << "--------------------------------------" << std::endl;
39
40      // Create a new graph
41      Graph undirectedMatrix;
42
43      // Add the vertices
44      undirectedMatrix.addVertex('A');
45      undirectedMatrix.addVertex('B');
46      undirectedMatrix.addVertex('C');
47      undirectedMatrix.addVertex('D');
48      undirectedMatrix.addVertex('E');
49
50      // Add the edges
51      undirectedMatrix.addEdge('A', 'B');
52      undirectedMatrix.addEdge('A', 'C');
53      undirectedMatrix.addEdge('A', 'D');
54      undirectedMatrix.addEdge('B', 'E');
55      undirectedMatrix.addEdge('C', 'D');
56      undirectedMatrix.addEdge('D', 'E');
57
```

```
58      // Display the adjacency matrix
59      undirectedMatrix.displayMatrix();
60
61      // ADJACENCY MATRIX OF WEIGHTED GRAPH
62      std::cout << "\nAdjacency Matrix of Weighted Graph:" << std::endl;
63      std::cout << "--------------------------------------" << std::endl;
64
65      // Create a new graph
66      Graph weightedMatrix;
67
```

```
68      // Add the vertices
69      weightedMatrix.addVertex('A');
70      weightedMatrix.addVertex('B');
71      weightedMatrix.addVertex('C');
72      weightedMatrix.addVertex('D');
73      weightedMatrix.addVertex('E');
74
75      // Add the weighted edges
76      weightedMatrix.addWeightedEdge('A', 'B', 4);
77      weightedMatrix.addWeightedEdge('A', 'C', 3);
78      weightedMatrix.addWeightedEdge('C', 'D', 2);
79      weightedMatrix.addWeightedEdge('D', 'B', 1);
80      weightedMatrix.addWeightedEdge('D', 'E', 1);
81      weightedMatrix.addWeightedEdge('E', 'A', 1);
82
83      // Display the weighted adjacency matrix
84      weightedMatrix.displayMatrix();
85
```

Output

```
Adjacency Matrix of Undirected Graph:
---------------------------------------
    A   B   C   D   E
A   0   1   1   1   0
B   1   0   0   0   1
C   1   0   0   1   0
D   1   0   1   0   1
E   0   1   0   1   0

Adjacency Matrix of Weighted Graph:
---------------------------------------
    A   B   C   D   E
A   0   4   3   0   0
B   0   0   0   0   0
C   0   0   0   2   0
D   0   1   0   0   1
E   1   0   0   0   0
```

Observations	The code successfully created the ***adjacency matrix*** for both the ***undirected graph*** and the ***weighted graph*** using the ***Graph class***. The ***addVertex()*** function was used to add the vertices ***A, B, C, D,*** and ***E,*** while the ***addEdge()*** function connected the vertices in the undirected graph by setting the corresponding values in the ***adjacency matrix*** to ***1*** in both directions. For the weighted graph, the ***addWeightedEdge()*** function was used to connect the vertices with their assigned weights, such as ***A to B with weight 4, A to C with weight 3, C to D with weight 2, D to B with weight 1, D to E with weight 1,*** and ***E to A with weight 1***. The ***displayMatrix()*** function used nested ***for loops*** to display the vertex labels and their corresponding matrix values, where ***0*** means there is no connection and the other numbers represent an edge or its weight. The output showed the correct matrix values based on the given graphs, with the ***undirected graph*** having ***connections in both directions*** and the ***weighted graph*** showing the ***correct direction and weight*** for each edge. Overall, the ***addVertex(), addEdge(), addWeightedEdge(),*** and ***displayMatrix()*** functions worked successfully and displayed both adjacency matrices correctly.

Table 10-2. Adjacency Matrix of Undirected Graph and Weighted Graph

A.3. Adjacency List

Implement the given code and indicate your output in the table.
(Table 10-3. Adjacency List of Undirected Graph and Directed Graph)

Screenshot of Code	**My Code: (Graph.h)**
	
	116 struct AdjNode{
	117 char vertex;
	118 AdjNode* next;
	119 };
	120
	121 class AdjacencyList{
	122 private:
	123 static const int maxVertices = 10;
	124 char vertices[maxVertices];
	125 AdjNode* head[maxVertices];
	126 int vertexCount;
	127
	128 void addNode(int index, char vertex);
	129
	130 public:
	131 AdjacencyList();

```
132         ~AdjacencyList();
133         void addVertex(char vertex);
134         int findVertex(char vertex);
135         void addUndirectedEdge(char start, char end);
136         void addDirectedEdge(char start, char end);
137         void displayList();
138 };
139
140 // Create a new adjacency list
141 AdjacencyList::AdjacencyList(){
142     vertexCount = 0;
143
144     // Initialize all head pointers
145     for(int i = 0; i < maxVertices; i++){
146         head[i] = nullptr;
147     }
148 }
149
150 // Delete the adjacency list
151 AdjacencyList::~AdjacencyList(){
152
153     // Delete all nodes
154     for(int i = 0; i < vertexCount; i++){
155         AdjNode* current = head[i];
156
157         while(current != nullptr){
158             AdjNode* temp = current;
159             current = current->next;
```

```
160             delete temp;
161         }
162     }
163 }
164
165 // Add a vertex
166 void AdjacencyList::addVertex(char vertex){
167
168     // Check if there is space for a new vertex
169     if(vertexCount < maxVertices){
170         vertices[vertexCount] = vertex;
171         head[vertexCount] = nullptr;
172         vertexCount++;
173     }
174 }
175
```

```cpp
// Find the position of a vertex
int AdjacencyList::findVertex(char vertex){

    // Search the vertices
    for(int i = 0; i < vertexCount; i++){
        if(vertices[i] == vertex){
            return i;
        }
    }

    return -1;
}

// Add a node to the adjacency list
void AdjacencyList::addNode(int index, char vertex){
    AdjNode* newNode = new AdjNode;

    newNode->vertex = vertex;
    newNode->next = nullptr;

    // Check if the list has no node
    if(head[index] == nullptr){
        head[index] = newNode;
    }
    else{

        // Start with the first node
        AdjNode* current = head[index];

        // Find the last node
        while(current->next != nullptr){
            current = current->next;
        }

        // Add the new node at the end
        current->next = newNode;
    }
}

// Add an undirected edge
void AdjacencyList::addUndirectedEdge(char start, char end){
    int startIndex = findVertex(start);
    int endIndex = findVertex(end);

    // Check if both vertices exist
```

```
221         if(startIndex != -1 && endIndex != -1){
222             addNode(startIndex, end);
223             addNode(endIndex, start);
224         }
225     }
226 
227     // Add a directed edge
228     void AdjacencyList::addDirectedEdge(char start, char end){
229         int startIndex = findVertex(start);
230 
231         // Check if the starting vertex exists
232         if(startIndex != -1){
233             addNode(startIndex, end);
234         }
235     }
236 
237     // Display the adjacency list
238     void AdjacencyList::displayList(){
239 
240         // Display all adjacency lists
241         for(int i = 0; i < vertexCount; i++){
242             std::cout << vertices[i] << " -> ";
243 
244             // Start with the first node
245             AdjNode* current = head[i];
246 
```

```
247             // Display the connected vertices
248             while(current != nullptr){
249                 std::cout << current->vertex;
250 
251                 if(current->next != nullptr){
252                     std::cout << " -> ";
253                 }
254 
255                 current = current->next;
256             }
257 
258             // Display NULL for an empty list
259             if(head[i] == nullptr){
260                 std::cout << "NULL";
261             }
262 
263             std::cout << std::endl;
264         }
265     }
```

My Code: (main_ILOA.cpp)

```
86      // ADJACENCY LIST OF UNDIRECTED GRAPH
87      std::cout << "\nAdjacency List of Undirected Graph:" << std::endl;
88      std::cout << "-------------------------------------" << std::endl;
89
90      // Create a new adjacency list
91      AdjacencyList undirectedList;
92
93      // Add the vertices
94      undirectedList.addVertex('A');
95      undirectedList.addVertex('B');
96      undirectedList.addVertex('C');
97      undirectedList.addVertex('D');
98      undirectedList.addVertex('E');
99
100     // Add the undirected edges
101     undirectedList.addUndirectedEdge('A', 'B');
102     undirectedList.addUndirectedEdge('A', 'C');
103     undirectedList.addUndirectedEdge('A', 'D');
104     undirectedList.addUndirectedEdge('B', 'E');
105     undirectedList.addUndirectedEdge('C', 'D');
106     undirectedList.addUndirectedEdge('D', 'E');
107
108     // Display the adjacency list
109     undirectedList.displayList();
110
```

```
111     // ADJACENCY LIST OF DIRECTED GRAPH
112     std::cout << "\nAdjacency List of Directed Graph:" << std::endl;
113     std::cout << "-----------------------------------" << std::endl;
114
115     // Create a new adjacency list
116     AdjacencyList directedList;
117
118     // Add the vertices
119     directedList.addVertex('A');
120     directedList.addVertex('B');
121     directedList.addVertex('C');
122     directedList.addVertex('D');
123     directedList.addVertex('E');
124
125     // Add the directed edges
126     directedList.addDirectedEdge('A', 'B');
127     directedList.addDirectedEdge('A', 'C');
128     directedList.addDirectedEdge('C', 'D');
129     directedList.addDirectedEdge('D', 'B');
130     directedList.addDirectedEdge('B', 'E');
131     directedList.addDirectedEdge('D', 'E');
```

```
132
133     // Display the adjacency list
134     directedList.displayList();
135     std::cout << std::endl;
136
137     return 0;
138 }
```

Output

```
Adjacency List of Undirected Graph:
----------------------------------------
A -> B -> C -> D
B -> A -> E
C -> A -> D
D -> A -> C -> E
E -> B -> D

Adjacency List of Directed Graph:
----------------------------------------
A -> B -> C
B -> E
C -> D
D -> B -> E
E -> NULL
```

Observations

The code successfully created the ***adjacency lists*** for both the ***undirected graph*** and the ***directed graph*** using the ***AdjacencyList class*** and the ***AdjNode structure***. The ***addVertex()*** function was used to add the vertices ***A, B, C, D,*** and ***E,*** while the ***findVertex()*** function searched for the position of each vertex before adding the connections. The ***addNode()*** function created new linked nodes and connected them at the end of each list, while the ***addUndirectedEdge()*** function added the connections in both directions, such as ***A to B*** and ***B to A***. For the directed graph, the ***addDirectedEdge()*** function added the connections based on their given directions, such as ***A to B, A to C, C to D, D to B, B to E,*** and ***D to E.*** The ***displayList()*** function used a ***while loop*** to go through each linked node and display the

	connected vertices, while ***nullptr*** was used to identify the end of a list and display ***NULL*** when a vertex had no outgoing connections, such as ***E*** in the directed graph. The ***~AdjacencyList() destructor*** also deleted the dynamically allocated nodes after use to help prevent memory leaks. The output showed the ***correct connections*** for both graphs, with the ***undirected graph*** displaying connections in ***both directions*** and the ***directed graph*** following the given ***edge directions***. Overall, the ***addVertex(), addNode(), addUndirectedEdge(), addDirectedEdge(),*** and ***displayList()*** functions worked successfully and displayed both adjacency lists correctly.

Table 10-3. Adjacency List of Undirected Graph and Directed Graph

My Output: (Complete)

Part 1

```
Create a Graph:
------------------------------------
Graph Vertices: A B C D E

Graph:
    A   B   C   D   E
A   0   1   1   1   0
B   1   0   0   0   1
C   1   0   0   1   0
D   1   0   1   0   1
E   0   1   0   1   0

Adjacency Matrix of Undirected Graph:
------------------------------------
    A   B   C   D   E
A   0   1   1   1   0
B   1   0   0   0   1
C   1   0   0   1   0
D   1   0   1   0   1
E   0   1   0   1   0
```

Part 2

```
Adjacency Matrix of Weighted Graph:
------------------------------------
    A   B   C   D   E
A   0   4   3   0   0
B   0   0   0   0   0
C   0   0   0   2   0
D   0   1   0   0   1
E   1   0   0   0   0

Adjacency List of Undirected Graph:
------------------------------------
A -> B -> C -> D
B -> A -> E
C -> A -> D
D -> A -> C -> E
E -> B -> D

Adjacency List of Directed Graph:
------------------------------------
A -> B -> C
B -> E
C -> D
D -> B -> E
E -> NULL
```

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Step 1. Include the required header files, as follows:
Step 2. Write the following struct in order to implement an edge in our graph:
Step 3. Next, overload the << operator for the graph so that it can be printed out using the following function:
Step 4. Implement the graph data structure that uses an edge list representation as follows:
Step 5. Now, we need a function to perform DFS for our graph. Implement it as follows:
Step 6. We shall test our implementation of the DFS on the graph shown here:

Use the following function to create and return the graph:

Note the use of null values for edge weights since DFS does not require edge weights. A simpler implementation of the graph could have omitted the edge weights entirely without affecting the behavior of our DFS algorithm.

Step 7. Finally, add the following test and driver code, which runs our DFS implementation and prints the output:
Step 8. Compile and run the preceding code

Include your output in the table.
(Table 10-4. Depth-First Search)

Screenshot of Code	**My Code: (GraphTraversalDFS.h)** ILOB > C GraphTraversalDFS.h > ... 1 #ifndef GRAPHTRAVERSALDFS_H 2 #define GRAPHTRAVERSALDFS_H 3 4 #include <iostream> 5 #include <vector> 6 #include <set> 7 #include <map> 8 #include <stack> 9 #include <utility> 10 #include <cstddef> 11 12 // Create a graph edge 13 template <typename T> 14 struct Edge{ 15 std::size_t src; 16 std::size_t dest;

```cpp

    // Compare edges using their weights
    bool operator<(const Edge<T> &e) const{
        return weight < e.weight;
    }

    bool operator>(const Edge<T> &e) const{
        return weight > e.weight;
    }
};

// Create a graph
template <typename T>
class Graph{
    private:
        std::size_t V;
        std::vector<Edge<T>> edge_list;

    public:
        Graph(std::size_t N);
        std::size_t vertices() const;
        const std::vector<Edge<T>> &edges() const;
        void add_edge(Edge<T> &&e);
        std::vector<Edge<T>> outgoing_edges(std::size_t v) const;
};
```

```cpp

// Create a new graph
template <typename T>
Graph<T>::Graph(std::size_t N) : V(N){
}

// Return the number of vertices
template <typename T>
std::size_t Graph<T>::vertices() const{
    return V;
}

// Return all graph edges
template <typename T>
const std::vector<Edge<T>> &Graph<T>::edges() const{
    return edge_list;
}

// Insert an edge into the graph
template <typename T>
void Graph<T>::add_edge(Edge<T> &&e){

```

```
65      // Check if both vertices exist
66      if(e.src >= 1 && e.src <= V &&
67         e.dest >= 1 && e.dest <= V){
68          edge_list.emplace_back(e);
69      }
70      else{
71          std::cerr << "Vertex out of bounds" << std::endl;
72      }
73  }
74
75  // Find the outgoing edges from a vertex
76  template <typename T>
77  std::vector<Edge<T>> Graph<T>::outgoing_edges(std::size_t v) const{
78      std::vector<Edge<T>> edges_from_v;
79
80      // Search the outgoing edges
81      for(const auto &e : edge_list){
82          if(e.src == v){
83              edges_from_v.emplace_back(e);
84          }
85      }
86
87      return edges_from_v;
88  }
89
```

```
90  // Display the graph
91  template <typename T>
92  std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
93
94      // Display each vertex and its outgoing edges
95      for(std::size_t i = 1; i <= G.vertices(); i++){
96          os << i << ":\t";
97          auto edges = G.outgoing_edges(i);
98
99          // Display the connected vertices
100         for(const auto &e : edges){
101             os << "{" << e.dest << ": " << e.weight << "}, ";
102         }
103
104         os << std::endl;
105     }
106
107     return os;
108 }
109
110 // Traverse the graph using Depth-First Search
111 template <typename T>
112 std::vector<std::size_t> depth_first_search(
113     const Graph<T> &G, std::size_t start){
```

```
114
115     std::stack<std::size_t> stack;
116     std::vector<std::size_t> visit_order;
117     std::set<std::size_t> visited;
118
119     // Check if the starting vertex exists
120     if(start < 1 || start > G.vertices()){
121         return visit_order;
122     }
123
124     // Start from the given vertex
125     stack.push(start);
126
127     // Continue while the stack is not empty
128     while(!stack.empty()){
129         auto current_vertex = stack.top();
130         stack.pop();
131
132         // Check if the vertex has not been visited
133         if(visited.find(current_vertex) == visited.end()){
134             visited.insert(current_vertex);
135             visit_order.push_back(current_vertex);
136
137             // Traverse the outgoing edges
138             for(const auto &e : G.outgoing_edges(current_vertex)){
139
```

```
140                 // Add unvisited vertices to the stack
141                 if(visited.find(e.dest) == visited.end()){
142                     stack.push(e.dest);
143                 }
144             }
145         }
146     }
147
148     return visit_order;
149 }
150
151 // Create the reference graph
152 template <typename T>
153 Graph<T> create_reference_graph(){
154     Graph<T> G(8);
155     std::map<unsigned, std::vector<std::pair<std::size_t, T>>> edges;
156
157     edges[1] = {{2, 0}, {5, 0}};
158     edges[2] = {{1, 0}, {5, 0}, {4, 0}};
159     edges[3] = {{4, 0}, {7, 0}};
160     edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
```

```
161     edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
162     edges[6] = {{4, 0}, {7, 0}, {8, 0}};
163     edges[7] = {{3, 0}, {6, 0}};
164     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
165 
166     // Insert the edges into the graph
167     for(const auto &i : edges){
168         for(const auto &j : i.second){
169             G.add_edge(Edge<T>{i.first, j.first, j.second});
170         }
171     }
172 
173     return G;
174 }
175 
176 // Test the DFS implementation
177 template <typename T>
178 void test_DFS(){
179 
180     // Create the reference graph
181     auto G = create_reference_graph<T>();
182 
183     // Display the graph
184     std::cout << G << std::endl;
185 
```

```
186     // Display the DFS traversal
187     std::cout << "DFS Order of vertices:" << std::endl;
188     std::cout << "----------------------------------------------"
189             << std::endl;
190 
191     auto dfs_visit_order = depth_first_search(G, 1);
192 
193     // Display the visited vertices
194     for(auto v : dfs_visit_order){
195         std::cout << v << std::endl;
196     }
197 }
198 
199 #endif
```

My Code: (main_DFS_ILOB.cpp)

ILOB > main_DFS_ILOB.cpp > ...

```cpp
#include <iostream>
#include "GraphTraversalDFS.h"

int main(){

    // Display the DFS activity heading
    std::cout << "\nDepth-First Search (DFS):" << std::endl;
    std::cout << "------------------------------------------------"
              << std::endl;

    // Run the DFS test
    using T = unsigned;
    test_DFS<T>();

    std::cout << std::endl;

    return 0;
}
```

Output

```
Depth-First Search (DFS):
------------------------------------------------
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
------------------------------------------------
1
5
8
6
7
3
4
2
```

Observations	The code successfully implemented the ***Depth-First Search (DFS) algorithm*** using the ***Graph class*** and the ***Edge* structure** to represent the vertices and connections of the graph. The ***create_reference_graph()*** function created the graph with ***8 vertices*** and added the connections using the ***add_edge() function***, while the ***operator<< overload*** displayed the vertices and their connected edges. The ***depth_first_search()*** function used a ***stack***, a ***set*** named ***visited***, and a vector named ***visit_order*** to keep track of the vertices that were visited. Since the ***stack follows the Last In, First Out (LIFO)*** rule, the last connected vertex added to the stack was processed first. The visited ***set*** also prevented the same vertex from being counted more than once, while the ***visit_order vector*** stored the order of the traversal. Starting from ***vertex 1***, the DFS traversal successfully visited the vertices in the order ***1, 5, 8, 6, 7, 3, 4,*** and ***2,*** which followed the order of the connections added to the graph and the way the stack processed them. The edge weights were set to ***0*** because the DFS algorithm did not need weights to decide which vertex to visit. Overall, the ***create_reference_graph(), add_edge(), operator<<,*** and ***depth_first_search()*** functions worked successfully and displayed the graph and its DFS traversal correctly.

Table 10-4. Depth-First Search

B.2. Breadth-First Search

Step 1: Include the required header files and declare the graph as follows:
Step 2: Write the following struct, which represents an edge in our graph:
Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:
Step 4: Write a class to define our graph data structure, as shown here:
Step 5: For this exercise, we shall test our implementation of BFS on the following graph:

We need a function to create and return the required graph. Note that while edge weights are assigned to each edge in the graph, this is not necessary since the BFS algorithm does not need to use edge weights. Implement the function as follows:

Step 6: Implement the breadth-first search like so:
Step 7: Add the following test and driver code that creates the reference graph, runs BFS starting from vertex 1, and outputs the results:

Include the output and observation in the table
(Table 10-5. Breadth-First Search)

Screenshot of Code	**My Code: (GraphTraversalBFS.h)**
	ILOB > C GraphTraversalBFS.h > ...
	1 #ifndef GRAPHTRAVERSALBFS_H
	2 #define GRAPHTRAVERSALBFS_H
	3
	4 #include <iostream>
	5 #include <vector>
	6 #include <set>
	7 #include <map>
	8 #include <queue>
	9 #include <utility>
	10 #include <cstddef>
	11
	12 // Create a graph edge
	13 template <typename T>
	14 struct Edge{
	15 std::size_t src;
	16 std::size_t dest;
	17 T weight;
	18
	19 // Compare edges using their weights
	20 bool operator<(const Edge<T> &e) const{

```
        return weight < e.weight;
    }

    bool operator>(const Edge<T> &e) const{
        return weight > e.weight;
    }
};

// Create a graph
template <typename T>
class Graph{
    private:
        std::size_t V;
        std::vector<Edge<T>> edge_list;

    public:
        Graph(std::size_t N);
        std::size_t vertices() const;
        const std::vector<Edge<T>> &edges() const;
        void add_edge(Edge<T> &&e);
        std::vector<Edge<T>> outgoing_edges(std::size_t v) const;
};

// Create a new graph
template <typename T>
Graph<T>::Graph(std::size_t N) : V(N){
}
```

```

// Return the number of vertices
template <typename T>
std::size_t Graph<T>::vertices() const{
    return V;
}

// Return all graph edges
template <typename T>
const std::vector<Edge<T>> &Graph<T>::edges() const{
    return edge_list;
}

// Insert an edge into the graph
template <typename T>
void Graph<T>::add_edge(Edge<T> &&e){

    // Check if both vertices exist
    if(e.src >= 1 && e.src <= V &&
       e.dest >= 1 && e.dest <= V){
        edge_list.emplace_back(e);
```

```
    }
    else{
        std::cerr << "Vertex out of bounds" << std::endl;
    }
}

// Find the outgoing edges from a vertex
template <typename T>
std::vector<Edge<T>> Graph<T>::outgoing_edges(std::size_t v) const{
    std::vector<Edge<T>> edges_from_v;

    // Search the outgoing edges
    for(const auto &e : edge_list){
        if(e.src == v){
            edges_from_v.emplace_back(e);
        }
    }

    return edges_from_v;
}

// Display the graph
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){

```

```
    // Display each vertex and its outgoing edges
    for(std::size_t i = 1; i <= G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);

        // Display the connected vertices
        for(const auto &e : edges){
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }

        os << std::endl;
    }

    return os;
}

// Create the reference graph
template <typename T>
Graph<T> create_reference_graph(){
    Graph<T> G(8);
    std::map<unsigned, std::vector<std::pair<std::size_t, T>>> edges;

```

```
116     edges[1] = {{2, 2}, {5, 3}};
117     edges[2] = {{1, 2}, {5, 5}, {4, 1}};
118     edges[3] = {{4, 2}, {7, 3}};
119     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
120     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
121     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
122     edges[7] = {{3, 3}, {6, 4}};
123     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
124
125     // Insert the edges into the graph
126     for(const auto &i : edges){
127         for(const auto &j : i.second){
128             G.add_edge(Edge<T>{i.first, j.first, j.second});
129         }
130     }
131
132     return G;
133 }
134
135 // Traverse the graph using Breadth-First Search
136 template <typename T>
137 std::vector<std::size_t> breadth_first_search(
138     const Graph<T> &G, std::size_t start){
139
140     std::queue<std::size_t> queue;
141     std::vector<std::size_t> visit_order;
142     std::set<std::size_t> visited;
143
```

```
144     // Check if the starting vertex exists
145     if(start < 1 || start > G.vertices()){
146         return visit_order;
147     }
148
149     // Start from the given vertex
150     queue.push(start);
151     visited.insert(start);
152
153     // Continue while the queue is not empty
154     while(!queue.empty()){
155         auto current_vertex = queue.front();
156         queue.pop();
157
158         // Visit the current vertex
159         visit_order.push_back(current_vertex);
160
```

```
161         // Traverse the outgoing edges
162         for(const auto &e : G.outgoing_edges(current_vertex)){
163 
164             // Add unvisited vertices to the queue
165             if(visited.find(e.dest) == visited.end()){
166                 visited.insert(e.dest);
167                 queue.push(e.dest);
168             }
169         }
170     }
171 
172     return visit_order;
173 }
174 
175 // Test the BFS implementation
176 template <typename T>
177 void test_BFS(){
178 
179     // Create the reference graph
180     auto G = create_reference_graph<T>();
181 
182     // Display the graph
183     std::cout << G << std::endl;
184 
```

```
185     // Display the BFS traversal
186     std::cout << "BFS Order of vertices:" << std::endl;
187     std::cout << "-----------------------------------------------"
188             << std::endl;
189 
190     auto bfs_visit_order = breadth_first_search(G, 1);
191 
192     // Display the visited vertices
193     for(auto v : bfs_visit_order){
194         std::cout << v << std::endl;
195     }
196 }
197 
198 #endif
```

My Code: (main_BFS_ILOB.cpp)

ILOB > main_BFS_ILOB.cpp > ...

```cpp
#include <iostream>
#include "GraphTraversalBFS.h"

int main(){

    // Display the BFS activity heading
    std::cout << "\nBreadth-First Search (BFS):" << std::endl;
    std::cout << "-----------------------------------------------"
              << std::endl;

    // Run the BFS test
    using T = unsigned;
    test_BFS<T>();

    std::cout << std::endl;

    return 0;
}
```

Output

```
Breadth-First Search (BFS):
-----------------------------------------------
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
-----------------------------------------------
1
2
5
4
8
3
6
7
```

Observations	The code successfully implemented the ***Breadth-First Search (BFS) algorithm*** using the ***Graph class*** and the ***Edge structure*** to represent the vertices and connections of the graph. The ***create_reference_graph()*** function created the graph with ***8 vertices*** and added the connections using the ***add_edge()*** function, while the ***operator<< overload*** displayed each vertex and its connected edges. The ***breadth_first_search()*** function used a ***queue***, a ***set*** named ***visited***, and a ***vector*** named ***visit_order*** to keep track of the traversal. The ***queue*** followed the ***First In, First Out (FIFO)*** rule, which allowed the algorithm to visit the starting vertex first, followed by its neighboring vertices before moving to the next level. The ***visited set*** prevented the same vertex from being added to the queue more than once, while the ***visit_order vector*** stored the order of the visited vertices. Starting from ***vertex 1***, the BFS traversal successfully visited the vertices in the order ***1, 2, 5, 4, 8, 3, 6,*** and ***7***, following the connections added in the ***create_reference_graph()*** function. Although edge weights were included when creating the graph, the ***breadth_first_search()*** function did not use them because BFS only needed the connections between vertices to determine the traversal order. Overall, the ***create_reference_graph(), add_edge(), operator<<,*** and ***breadth_first_search()*** functions worked successfully and displayed the graph and its BFS traversal correctly.

Table 10-5. Breadth-First Search

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

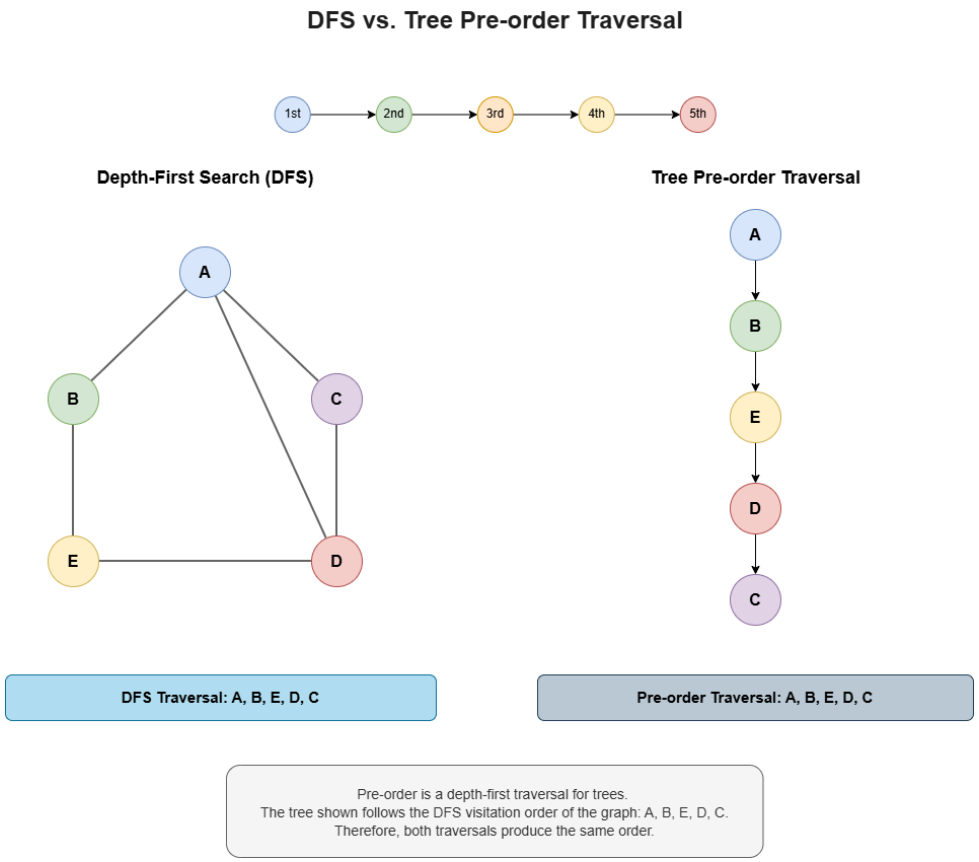
1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.**

    - *The most helpful algorithm for this situation is Depth-First Search (DFS) because the method allows the person to explore one path first until reaching the end before going back to the previous location and checking other possible paths. The algorithm uses a stack to keep track of which location to visit next and a visited set to prevent visiting the same location more than once. This method is useful when the goal is to explore all connected locations by going deeper into one path before moving to another path.*

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**

My Work:

Graphical Comparison



Pseudocode

DFS Traversal:

```
FUNCTION DFS(G, start)
    CREATE an empty stack
    CREATE an empty visit_order list
    CREATE an empty visited set

    IF start does not exist in G THEN
        RETURN visit_order
    END IF
```

```
    PUSH start onto the stack

    WHILE stack is not empty DO
        SET current_vertex = top vertex popped from the stack

        IF current_vertex is not in visited THEN
            ADD current_vertex to visited
            ADD current_vertex to visit_order

            FOR each vertex in G, in reverse order DO
                IF there is an edge between current_vertex and vertex
                AND vertex is not in visited THEN
                    PUSH vertex onto the stack
                END IF
            END FOR
        END IF
    END WHILE

    RETURN visit_order
END FUNCTION
```

Pre-order Traversal:

```
FUNCTION PreOrder(node)
    IF node is not NULL THEN
        PRINT node's key

        IF node has a first child THEN
            CALL PreOrder(node's first child)
        END IF

        IF node has a next sibling THEN
            CALL PreOrder(node's next sibling)
        END IF
    END IF
END FUNCTION
```

main_ILOC.cpp

```
START
   CREATE a graph G

   ADD vertices A, B, C, D, E to G

   ADD edges A-B, A-C, A-D, B-E, C-D, D-E

   SET result = DFS(G, A)

   PRINT "DFS Traversal:"
   PRINT separator line

   FOR each vertex in result DO
      PRINT vertex
   END FOR

   CREATE a GeneralTree root node with key A

   CREATE tree nodes B, E, D, C

   INSERT B as a child of root
   INSERT E as a child of B
   INSERT D as a child of E
   INSERT C as a child of D

   PRINT "Pre-order Traversal:"
   PRINT separator line

   CALL PreOrder(root)

   DELETE the tree

END
```

Screenshot of Code

My Code: (GeneralTree.h)

ILOC > C GeneralTree.h > ...

```
1   #ifndef GENERALTREE_H
2   #define GENERALTREE_H
3
4   #include <iostream>
5
6   template <typename T>
7   class GeneralTree{
8       private:
9           T key;
10          GeneralTree<T>* firstchild;
11          GeneralTree<T>* sibling;
12
13      public:
14          GeneralTree(T value);
15          ~GeneralTree();
16
17          void insert(GeneralTree<T>* child);
18          void preOrder();
19  };
20
```

```
21  // Create a new tree node
22  template <typename T>
23  GeneralTree<T>::GeneralTree(T value){
24      key = value;
25      firstchild = nullptr;
26      sibling = nullptr;
27  }
28
29  // Delete the child and sibling nodes
30  template <typename T>
31  GeneralTree<T>::~GeneralTree(){
32      delete firstchild;
```

```
33      delete sibling;
34  }
35
36  // Insert a child node
37  template <typename T>
38  void GeneralTree<T>::insert(GeneralTree<T>* child){
39
40      // Check if the node has no child
41      if(firstchild == nullptr){
42          firstchild = child;
43      }
44      else{
45
46          // Start with the first child
47          GeneralTree<T>* current = firstchild;
48
49          // Find the last sibling
50          while(current->sibling != nullptr){
51              current = current->sibling;
52          }
53
54          // Add the child as the last sibling
55          current->sibling = child;
56      }
57  }
58
```

```
59  // Traverse the tree using Pre-order
60  template <typename T>
61  void GeneralTree<T>::preOrder(){
62
63      // Visit the current node
64      std::cout << key << " ";
65
66      // Traverse the first child and its descendants
67      if(firstchild != nullptr){
68          firstchild->preOrder();
```

```
69          }
70
71          // Traverse the remaining siblings
72          if(sibling != nullptr){
73              sibling->preOrder();
74          }
75      }
76
77      #endif
```

My Code: (Graph.h)

ILOC > C Graph.h > ...

```
1   #ifndef GRAPH_H
2   #define GRAPH_H
3
4   #include <iostream>
5
6   class Graph{
7       private:
8           static const int maxVertices = 10;
9           char vertices[maxVertices];
10          int matrix[maxVertices][maxVertices];
11          int vertexCount;
12
13      public:
14          Graph();
15          void addVertex(char vertex);
16          int findVertex(char vertex) const;
17          void addEdge(char start, char end);
18          void addWeightedEdge(char start, char end, int weight);
19          void displayVertices();
20          void displayMatrix();
21
22          int getVertexCount() const;
23          char getVertexAt(int index) const;
24          bool hasEdge(char start, char end) const;
25  };
26
27  // Create a new graph
28  inline Graph::Graph(){
29      vertexCount = 0;
30
```

```
    // Initialize the adjacency matrix
    for(int i = 0; i < maxVertices; i++){
        for(int j = 0; j < maxVertices; j++){
            matrix[i][j] = 0;
        }
    }
}

// Add a vertex
inline void Graph::addVertex(char vertex){

    // Check if there is space for a new vertex
    if(vertexCount < maxVertices && findVertex(vertex) == -1){
        vertices[vertexCount] = vertex;
        vertexCount++;
    }
}

// Find the position of a vertex
inline int Graph::findVertex(char vertex) const{
    for(int i = 0; i < vertexCount; i++){
        if(vertices[i] == vertex){
            return i;
        }
    }

    return -1;
}
```

```

// Get the number of vertices
inline int Graph::getVertexCount() const{
    return vertexCount;
}

// Get a vertex by its index
inline char Graph::getVertexAt(int index) const{
    if(index >= 0 && index < vertexCount){
        return vertices[index];
    }

    return '\0';
}

// Check if an edge exists
inline bool Graph::hasEdge(char start, char end) const{
    int startIndex = findVertex(start);
    int endIndex = findVertex(end);

```

```
    if(startIndex != -1 && endIndex != -1){
        return matrix[startIndex][endIndex] != 0;
    }

    return false;
}

// Add an undirected edge
inline void Graph::addEdge(char start, char end){
    int startIndex = findVertex(start);
    int endIndex = findVertex(end);

    // Check if both vertices exist
    if(startIndex != -1 && endIndex != -1){
        matrix[startIndex][endIndex] = 1;
        matrix[endIndex][startIndex] = 1;
    }
}

// Add a weighted directed edge
inline void Graph::addWeightedEdge(char start, char end, int weight){
    int startIndex = findVertex(start);
    int endIndex = findVertex(end);

    // Check if both vertices exist
    if(startIndex != -1 && endIndex != -1){
        matrix[startIndex][endIndex] = weight;
```

```
    }
}

// Display the graph vertices
inline void Graph::displayVertices(){
    for(int i = 0; i < vertexCount; i++){
        std::cout << vertices[i] << " ";
    }

    std::cout << std::endl;
}

// Display the adjacency matrix
inline void Graph::displayMatrix(){
    std::cout << "    ";

    for(int i = 0; i < vertexCount; i++){
        std::cout << vertices[i] << "   ";
    }

```

```
126     std::cout << std::endl;
127
128     for(int i = 0; i < vertexCount; i++){
129         std::cout << vertices[i] << "   ";
130
131         for(int j = 0; j < vertexCount; j++){
132             std::cout << matrix[i][j] << "   ";
133         }
134
135         std::cout << std::endl;
136     }
137 }
138
139 #endif
```

My Code: (Problem2.h)

```
ILOC > C Problem2.h > ...
 1  #ifndef PROBLEM2_H
 2  #define PROBLEM2_H
 3
 4  #include <stack>
 5  #include <vector>
 6  #include <set>
 7  #include "GeneralTree.h"
 8  #include "Graph.h"
 9
10  // Traverse the graph using DFS
11  inline std::vector<char> depth_first_search(const Graph &G, char start){
12      std::stack<char> stack;
13      std::vector<char> visit_order;
14      std::set<char> visited;
15
16      // Check if the starting vertex exists
17      if(G.findVertex(start) == -1){
18          return visit_order;
19      }
20
21      stack.push(start);
22
23      while(!stack.empty()){
24          char current_vertex = stack.top();
25          stack.pop();
26
27          if(visited.find(current_vertex) == visited.end()){
28              visited.insert(current_vertex);
29              visit_order.push_back(current_vertex);
```

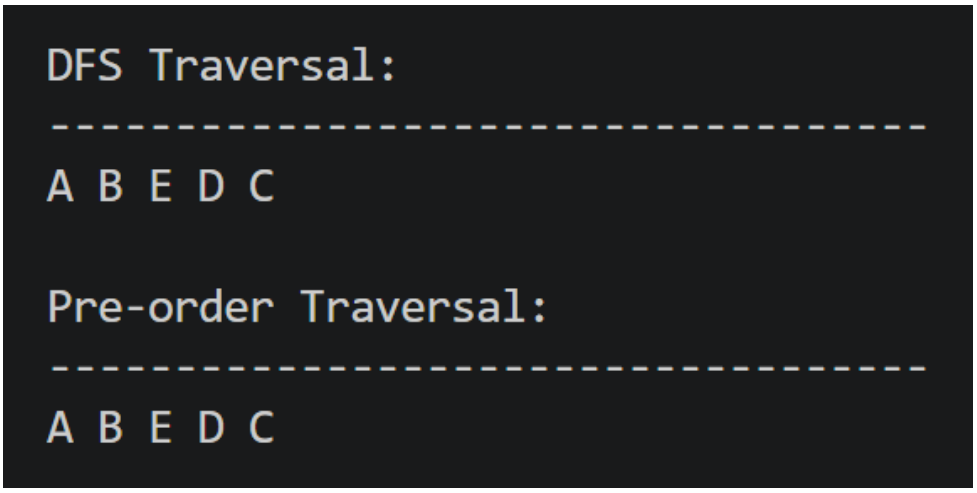
```
30
31              // Push neighbors in reverse order
32              for(int i = G.getVertexCount() - 1; i >= 0; i--){
33                  char neighbor = G.getVertexAt(i);
34
35                  if(G.hasEdge(current_vertex, neighbor) &&
36                     visited.find(neighbor) == visited.end()){
37                      stack.push(neighbor);
38                  }
39              }
40          }
41      }
42
43      return visit_order;
44  }
45
46  #endif
```

My Code: (main_ILOC.cpp)

```
ILOC > main_ILOC.cpp > main()
1   #include <iostream>
2   #include <vector>
3   #include "Problem2.h"
4
5   int main(){
6
7       // Create the graph
8       Graph G;
9
10      // Insert vertices into the graph
11      G.addVertex('A');
12      G.addVertex('B');
13      G.addVertex('C');
14      G.addVertex('D');
15      G.addVertex('E');
16
17      // Insert edges into the graph
18      G.addEdge('A', 'B');
19      G.addEdge('A', 'C');
20      G.addEdge('A', 'D');
21      G.addEdge('B', 'E');
22      G.addEdge('C', 'D');
23      G.addEdge('D', 'E');
24
25      // Display the DFS traversal
26      std::vector<char> result = depth_first_search(G, 'A');
```

```
27
28      std::cout << "\nDFS Traversal:" << std::endl;
29      std::cout << "---------------------------------" << std::endl;
30
31      for(char vertex : result){
32          std::cout << vertex << " ";
33      }
34
35      std::cout << std::endl;
36
37      // Create the root node
38      GeneralTree<char>* root = new GeneralTree<char>('A');
39
40      // Insert values into the tree
41      GeneralTree<char>* B = new GeneralTree<char>('B');
42      GeneralTree<char>* E = new GeneralTree<char>('E');
43      GeneralTree<char>* D = new GeneralTree<char>('D');
44      GeneralTree<char>* C = new GeneralTree<char>('C');
45
46      root->insert(B);
47      B->insert(E);
48      E->insert(D);
49      D->insert(C);
50
```

```
51      // Display the Pre-order traversal
52      std::cout << "\nPre-order Traversal:" << std::endl;
53      std::cout << "---------------------------------" << std::endl;
54      root->preOrder();
55      std::cout << std::endl << std::endl;
56
57      // Delete the tree
58      delete root;
59
60      return 0;
61  }
```

Console Output	
Answer	The code successfully demonstrated the relationship between ***Depth-First Search (DFS)*** and ***Pre-order Traversal*** by using the ***Graph class***, the ***GeneralTree class***, and the ***depth_first_search()*** function. The ***Graph.h*** file used an ***adjacency matrix*** to store the vertices and connections, while the ***addVertex()*** and ***addEdge()*** functions added the vertices and edges needed for the graph. The ***Problem2.h*** file implemented the DFS algorithm using a ***stack***, a ***set*** named ***visited***, and a ***vector*** named ***visit_order***. The stack followed the ***Last In, First Out (LIFO)*** rule, which allowed the algorithm to explore one path first before going back and checking the other connected vertices. The ***visited set*** prevented the same vertex from being processed more than once, while the ***visit_order vector*** stored the order of the traversal. Meanwhile, the ***GeneralTree.h*** file implemented the ***GeneralTree*** class using the ***firstchild*** and ***sibling*** pointers, and the ***preOrder()*** function visited the current node first before moving to its first child and next sibling. In ***main_ILOC.cpp***, the graph was created with vertices ***A, B, C, D,*** and ***E,*** and the required edges were added before running DFS starting from ***A***. A tree was also created with the nodes arranged as ***A, B, E, D,*** and ***C***, allowing the ***Pre-order Traversal*** to follow the same visiting order for comparison. The output successfully displayed ***A, B, E, D, and C*** for both traversals, showing that the two methods produced the same order in this specific example. However, DFS and Pre-order Traversal do not always produce the same order because DFS is used for graphs while Pre-order Traversal follows the structure of a tree. Overall, the ***graphical comparison, pseudocode,*** and ***code implementation*** worked successfully and showed how DFS explores connected vertices and how Pre-order Traversal visits the nodes of a tree.

3. **In the performed code, what data structure is used to implement the Breadth First Search?**

   - *The Breadth-First Search (BFS) algorithm uses a queue to keep track of the vertices that need to be visited next. The queue follows the First In, First Out (FIFO) rule, which means the vertex added first will be processed first. This allows the algorithm to visit the neighboring vertices level by level before moving farther away from the starting vertex.*

4. **How many times can a node be visited in the BFS?**

   - *Each node should only be visited once in the Breadth-First Search (BFS) algorithm because the visited set keeps track of the nodes that have already been processed and prevents them from being visited repeatedly. This helps the algorithm avoid unnecessary repeated visits and allows the traversal to explore the connected vertices in an organized way.*

Conclusion:

The activity helped in understanding how graphs work and how they can be implemented using C++. For the lessons learned, an adjacency matrix stores the connections between vertices using a matrix, while an adjacency list stores the connected vertices using linked nodes. The activity also showed how DFS uses a stack to explore one path before backtracking, while BFS uses a queue to visit neighboring vertices level by level. For the procedure, the codes were created and tested step by step, starting with adding vertices and edges, then displaying the graph using an adjacency matrix and adjacency list, and finally applying DFS and BFS. The important parts such as addVertex(), addEdge(), addWeightedEdge(), displayMatrix(), displayList(), depth_first_search(), and breadth_first_search() helped show how each graph operation and traversal method works. For the supplementary activity, the relationship between DFS and Pre-order Traversal was examined through a graphical comparison, pseudocode, and code implementation, showing how both methods produced the same visiting order in the given example.

One difficulty faced was understanding how the adjacency matrix and adjacency list store connections differently, especially when dealing with directed, undirected, and weighted graphs. Another difficulty was keeping track of the visiting order in DFS and BFS because the stack and queue follow different rules. Checking the edge directions, matrix values, and traversal results also needed extra attention to make sure the output matched the expected answers. Even with these difficulties, the codes and results were completed based on the required graph operations and traversal methods. Overall, the activity was done well because the concepts were understood and successfully applied in the code, but there is still room for improvement in tracing graph connections, understanding traversal step by step, and becoming more familiar with the functions without depending too much on examples. More practice with different graph structures and traversal problems can help improve understanding and confidence in implementing these algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.